

1 - Components & Styling

In this section we're going to start to put our UI together. So the way that we setup our layout and used partials is kind of the old way of doing things. Instead of partials, we want to setup our UI with components. This works in a similar way to frontend JavaScript frameworks like React. We'll have a header component, a hero component. We're even going to make the navigation links components. And these components can be passed props, which are basically custom attributes. Again, if you've worked with something like React, this will be a very familiar concept. We'll learn about slots, which are another way to make the components dynamic.

We're also going to setup Tailwind CSS because we'll be using Tailwind classes for our styling. We'll be adding a tiny bit of frontend JavaScript for our mobile menu toggle, so we'll also learn how to use public assets like JavaScript and CSS files as well as images.

2 - Intro To Components

Components were introduced in Laravel 7.0. Components provide a way to encapsulate HTML, CSS, and JavaScript into reusable pieces that can be used throughout your application. They support both class-based components (using PHP classes) and anonymous components (using simple Blade templates). This helps in keeping your views organized and maintainable. If you are coming from a frontend JavaScript framework like React or Vue, you will feel right at home with Laravel components.

In this chapter, we will learn how to create and use components in Laravel.

Create A Component

Let's create a component that we can use in our views. Run the following command to create a new component:

```
php artisan make:component Header
```

This command will create a new component class at `app/View/Components/Header.php` as well as a Blade template at `resources/views/components/header.blade.php`. For the most part, you will work in the template because this is where you will put the HTML for the component. But let's take a look at the component class first.

- It has a namespace and some imports that are needed for the class to function properly.
- The class extends `Component`, which is a base class that provides some useful methods and properties for the component.
- The `__construct` method is where you can pass data into your component. If your component needs any initialization or data when it's created, you can do that here. For example, you could add parameters to the constructor and assign them to class properties.

- The `render()` is a method that returns the view associated with this component. This method tells Laravel which Blade view file to use when rendering the component.

For most components, we won't need to do much here, but I want you to know what it does.

Open the view at `resources/views/components/header.blade.php`. This is where you will put the HTML for the component.

Add the following code for now:

```
<nav>
  <a href="/">Home</a>
  <a href="/jobs">Jobs</a>
  <a href="/jobs/create">Create Job</a>
</nav>
```

Use The Component

Now that we have a component, let's use it in our layout. Open the `layout.blade.php` file and replace the `<h1>` tag with the following code:

```
<x-header />
```

We use the `x-` prefix to tell Laravel that we want to use a component. The `x-` prefix is a convention that is used to indicate that the tag is a component.

Now you can delete the `partials/header.blade.php` file because we are using the component instead. You can delete the entire `partials` directory.

Sub-Folders

You can also create sub-folders inside the `components` directory. For example, let's create a folder called `inputs` and create a file called `text.blade.php`. This is something that we will be doing later, so we can create the folder and file now.

In the `text.blade.php` file, add the following code:

```
<input type="text" />
```

Now, let's use this component in our layout just to see how we use it. Open the `layout.blade.php` file and replace the `<input>` tag with the following code:

```
<x-inputs.text />
```

So we add the `x-` prefix then the name of the folder and the file name. This is how you reference components in sub-folders.

You can delete this line now. But keep the folder and file because we will be using it later.

In the next lesson, I will show you how to use components for your layout.

3 - Layout Components & Slots

As I said earlier, there are two ways to work with layouts. There is the template inheritance method, and there is the component method. We have already seen how to use the template inheritance method. In this lesson, we will learn how to use components for our layout and ultimately that is what we will use for our project.

Let's create a new Layout component. Run the following command to create a new component:

```
php artisan make:component Layout
```

Add the following to `resources/views/components/layout.blade.php`:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Workopia</title>
  </head>

  <body class="bg-gray-100">
    <x-header />
    <main class="container mx-auto p-4 mt-4">{{ $slot }}</main>
  </body>
</html>
```

The changes that we made are instead of `@yield`, we are using `{{ $slot }}`. The `$slot` variable is a special variable that contains the content of the child view. This is how we pass the content from the child view to the layout component. We can have multiple slots in a component by using named slots, but in this case, we only have one.

Updating The Views

Now in order to use the layout component, we need to update both the `index.blade.php` and `create.blade.php` files.

Here is the new code for `index.blade.php`:

```
<x-layout>
  <ul>
    @forelse($jobs as $job)
      <li>{{ $job }}</li>
    @empty
      <li>No jobs found</li>
    @endforelse
  </ul>
</x-layout>
```

We wrap everything in file with `<x-layout>`. This is cleaner than using `@extends` and `@section`.

Now update the `home.blade.php` and `create.blade.php` files with the following code respectively:

```
<x-layout>
  <h1>Welcome to Workopia</h1>
  <p>Find your dream job today</p>
</x-layout>
```

```
<x-layout>
  <x-slot name="title"> Create Job </x-slot>
  <h1>Create Job</h1>
  <form action="/jobs" method="POST">
    @csrf
    <input type="text" name="title" placeholder="Title" />
    <input type="text" name="description" placeholder="Description" />
    <button type="submit">Submit</button>
  </form>
</x-layout>
```

We do the same thing here. We wrap everything in the file with `<x-layout>`. We also set the title of the create page using the `<x-slot>` directive.

Now everything should work as before. You can test it by visiting the `/jobs` and `/jobs/create` routes.

From now on, we will use the component method for our layout. It is cleaner and more organized than the template inheritance method. Everything else as far as the controller and passing the data to the view remains the same.

You can delete the `layout.blade.php` file in the `views` folder with the template inheritance method. We won't be using it anymore.

Move The Layout Component

If you want to move the layout component from the components folder to the views folder, you can. I am going to do that now. I am going to move the `layout.blade.php` file from `resources/views/components` to `resources/views/layouts`.

This will now break because it is looking in the components folder for the layout component. We can fix this by updating the `render` method in the `app/Views/Layout.php` file to the following:

```
public function render()  
{  
    return view('layout');  
}
```

Now you can clear the view cache by running the following command:

```
php artisan view:clear
```

Now it should work as expected.

Favicon

I have attached a download for a favicon that you can use for the project. You can download it from the resources section of this lesson. With Laravel, you simply add the file to the `public` folder and do a hard refresh of the browser to see the changes. Of course, if you want to use your own favicon, you can do that as well.

haha No problem at all. No time requirements to be a mod. That David guy is 100% insane. I had to block him. He used the code from my old PHP course as a base for Trongate. Which is fine. I said that's great. But then when I said I wouldn't promote it on my channel he acted like I stabbed him in the back. Very strange guy. Like seriously, I think he is mentally ill.

4 - Tailwind CSS & Vite Hot Reloading

Now we are going to set up Tailwind CSS in our Laravel project. Tailwind CSS is a utility-first CSS framework that is easy to use and customize. It is a great choice for building modern and responsive web applications.

There are a few ways to implement Tailwind. We can use the CDN, but that's not really recommended for production. We're going to install Tailwind Using NPM. This way, we can customize the configuration and build the CSS file. So the CSS will only include the classes that we are using, which will make the file size smaller. Also, we're going to be using Vite as our frontend dev tool which includes hot reloading. Up to this point, if you're using Laravel Herd or something other than the Artisan server, we've had to refresh manually after every change. After setting up Vite and running the server, it should refresh right when we save a file, which is nice.

Tailwind Setup Using NPM

If you go to <https://tailwindcss.com/docs/guides/laravel> there are instructions on how to set up Tailwind in a Laravel project. We will follow along with these instructions.

NPM stands for Node Package Manager. It is a package manager for JavaScript. It is used to install and manage packages for a project. If you are coming from the JavaScript world, you are probably already familiar with NPM. If not, don't worry, it is easy to use.

You do need to install Node.js to use NPM. You can download it from <https://nodejs.org/en/>. Once you have Node.js installed, you will have NPM installed as well.

Install Tailwind CSS

First, open a terminal window up in the project directory. Run the following command to install Tailwind CSS:

```
npm install -D tailwindcss postcss autoprefixer
```

Now, generate the Tailwind configuration file by running the following command:

```
npx tailwindcss init -p
```

Open the `tailwind.config.js` file and add the following code:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './resources/**/*.blade.php',
    './resources/**/*.js',
    './resources/**/*.vue',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
};
```

This will tell Tailwind to look for classes in the Blade files, JavaScript files, and Vue files. I know we are not using Vue at this point, but it is good to have it set up in case we decide to use it later. Vue and Laravel are often used together.

Now add the following code to the `resources/css/app.css` file:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

This will import the Tailwind CSS styles into our project.

Now we just need to include the CSS file in the layout. Open the `resources/views/components/layout.blade.php` file and add the following code just above the title:

```
@vite('resources/css/app.css')
```

Build Tailwind CSS

Now we need to build the Tailwind CSS file. We can have it watch for changes by running the following command:

```
npm run dev
```

So from now on, you will run `npm run dev` to build the Tailwind CSS file. This will watch for changes and rebuild the file when changes are made. You also have hot reloading available.

6 - Header Component & `url()` Helper

We are going to add the content and styling to the header component and also look at the `url()` helper.

We already have our Header component at `resources/views/components/header.blade.php`. Let's update this file.

This will be the first time that we copy some HTML from the theme files. I put my theme files, right in the root of the project. You can put them anywhere you want. We want to open `_theme_files/index.html` and copy the `<header>` section and everything inside it. It should look like this:

```
<header class="bg-blue-900 text-white p-4">
  <div class="container mx-auto flex justify-between items-center">
    <h1 class="text-3xl font-semibold">
      <a href="index.html">Workopia</a>
    </h1>
    <nav class="hidden md:flex items-center space-x-4">
      <a href="jobs.html" class="text-white hover:underline py-2">All Jobs</a>
      <a href="saved-jobs.html" class="text-white hover:underline py-2">
        >Saved Jobs</a>
      >
      <a href="login.html" class="text-white hover:underline py-2">Login</a>
      <a href="register.html" class="text-white hover:underline py-2">
        >Register</a>
      >
      <a href="dashboard.html" class="text-white hover:underline py-2">
        <i class="fa fa-gauge mr-1"></i> Dashboard
      </a>
      <a
        href="create-job.html"
        class="bg-yellow-500 hover:bg-yellow-600 text-black px-4 py-2 rounded
```

```

    hover:shadow-md transition duration-300"
    >
        <i class="fa fa-edit"></i> Create Job
    </a>
</nav>
<button id="hamburger" class="text-white md:hidden flex items-center">
    <i class="fa fa-bars text-2xl"></i>
</button>
</div>
<!-- Mobile Menu -->
<div
    id="mobile-menu"
    class="hidden md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
>
    <a href="jobs.html" class="block px-4 py-2 hover:bg-blue-700">All Jobs</a>
    <a href="saved-jobs.html" class="block px-4 py-2 hover:bg-blue-700">
        >Saved Jobs</a>
    >
    <a href="dashboard.html" class="block px-4 py-2 hover:bg-blue-700">
        >Dashboard</a>
    >
    <a href="login.html" class="block px-4 py-2 hover:bg-blue-700">Login</a>
    <a href="register.html" class="block px-4 py-2 hover:bg-blue-700">
        >Register</a>
    >
    <a
        href="create-job.html"
        class="block px-4 py-2 bg-yellow-500 hover:bg-yellow-600 text-black"
    >
        <i class="fa fa-edit"></i> Create Job
    </a>
</div>
</header>

```

Paste it within the `/resources/views/components/header.blade.php` file.

Right, now it has all of the links. Some of them will only show when logged in or logged out, but we will get to that later. Right now, I just want to focus on the header being displayed and styled.

If you go to the homepage now, you should see the Header at the top of the page. It is missing the icons and the mobile dropdown menu does not work because it needs a little JavaScript. We will get to that in a bit.

url() Helper

We have a few links in the Header. We need to update them to use the `url()` helper. This is a Laravel helper that generates a URL for the given path. This is useful because it will generate the correct URL for the current environment. For example, if you are in a local environment, it will generate `http://localhost:8000/jobs`. If you are in a production environment, it will generate `https://workopia.com/jobs`.

Update the links in the Header to use the `url()` helper. Here is an example:

Change this:

```
<a href="jobs.html" class="text-white hover:underline py-2">All Jobs</a>
```

To this:

```
<a href="{{ url('/jobs') }}" class="text-white hover:underline py-2"
  >All Jobs</a>
>
```

Use the slash before the route name. Otherwise, you could get urls like

<http://localhost:8000/jobs/jobs>.

Do this for all the links in the Header. Here are the links:

- All Jobs - `url('/jobs')`
- Saved Jobs - `url('/jobs/saved')`
- Dashboard - `url('/dashboard')`
- Create Job - `url('/jobs/create')`
- Login - `url('/login')`
- Register - `url('/register')`

Now you can navigate to the different pages using the Header links. Most of them will not work obviously because we have not created the routes yet. There is also a `route()` helper for named routes. We will use that for most other links but for the navigation links, we will use the `url()` helper.

The code should now look like this:

```
<header class="bg-blue-900 text-white p-4">
  <div class="container mx-auto flex justify-between items-center">
    <h1 class="text-3xl font-semibold">
      <a href="{{ url('/') }}">Workopia</a>
    </h1>
    <nav class="hidden md:flex items-center space-x-4">
      <a href="{{ url('/jobs') }}" class="text-white hover:underline py-2"
        >All Jobs</a>
      <
      <a href="{{ url('/jobs/saved') }}" class="text-white hover:underline py-2"
        >Saved Jobs</a>
      <
      <a href="{{ url('/login') }}" class="text-white hover:underline py-2"
        >Login</a>
      <
      <a href="{{ url('/register') }}" class="text-white hover:underline py-2"
        >Register</a>
      <
      <a href="{{ url('/dashboard') }}" class="text-white hover:underline py-2">
        <i class="fa fa-gauge mr-1"></i> Dashboard
      </a>
```

```

        <a
            href="{{ url('/jobs/create') }}"
            class="bg-yellow-500 hover:bg-yellow-600 text-black px-4 py-2 rounded
            hover:shadow-md transition duration-300"
        >
            <i class="fa fa-edit"></i> Create Job
        </a>
    </nav>
    <button id="hamburger" class="text-white md:hidden flex items-center">
        <i class="fa fa-bars text-2xl"></i>
    </button>
</div>
<!-- Mobile Menu -->
<div
    id="mobile-menu"
    class="hidden md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
>
    <a href="{{ url('/jobs') }}" class="block px-4 py-2 hover:bg-blue-700"
        >All Jobs</a>
    >
    <a href="{{ url('/jobs/saved') }}" class="block px-4 py-2 hover:bg-blue-700"
        >Saved Jobs</a>
    >
    <a href="{{ url('/dashboard') }}" class="block px-4 py-2 hover:bg-blue-700"
        >Dashboard</a>
    >
    <a href="{{ url('/login') }}" class="block px-4 py-2 hover:bg-blue-700"
        >Login</a>
    >
    <a href="{{ url('/register') }}" class="block px-4 py-2 hover:bg-blue-700"
        >Register</a>
    >
    <a
        href="{{ url('/jobs/create') }}"
        class="block px-4 py-2 bg-yellow-500 hover:bg-yellow-600 text-black"
    >
        <i class="fa fa-edit"></i> Create Job
    </a>
</div>
</header>

```

Now your links should show on larger screens. We will get to the mobile links soon.

Adding Font Awesome

Let's open the `views/components/layout.blade.php` file and add the following code to the `<head>` tag:

```

<link
    rel="stylesheet"
    href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.6.0/css/all.min.css"
    integrity="sha512-

```

```
Kc323vGBEqzTmouAECnVceyQqyqdsSiqLQISBL29aUW4U/M7pSPA/gEUZQqv1cwx40nYxTxve5UMg5GT6L4JJg=="
  crossorigin="anonymous"
  referrerpolicy="no-referrer"
/>
```

Now the icons should show.

7 - Conditional Classes & @php Directive

In this lesson, we are going to look at the `@php` directive, conditional classes as well as the `request()` helper function.

@php Directive

The `@php` directive allows you to execute PHP code within your Blade templates. This can be useful when you need to perform some logic that is not possible with the Blade syntax.

Open the `resources/views/components/header.blade.php` file and add this to the very top:

```
@php
    echo 'Hello, World!';
@endphp
```

You should see `Hello, World!` printed at the top of the page.

request() Helper Function

Change the hello world to the following:

```
@php
    $isActive = request()->is('/');
    echo $isActive ? 'active' : '';
@endphp
```

Here, we are using the `request()` helper function to check if the current URL is the homepage. If it is, we set the `$isActive` variable to `true`. We then use a ternary operator to print `active` if `$isActive` is `true`.

You can delete the whole `@php` block for now. I just wanted to show you how to use it.

Conditional Classes

I want to make the active or current link bold and yellow. Right now, the only link that works is the `/jobs` link. Let's add an additional link for the homepage.

```
<a href="{{ url('/') }}" class="text-white hover:underline py-2">Home</a>
```

There are many ways to do conditional classes. We can add a ternary in the class attribute like so:

```
<a href="/" class="{{ request()->is('/') ? 'font-bold text-yellow-500' : '' }}">Home</a>
```

Here are all of the links:

```
<a href="{{ url('/') }}"
  class="text-white hover:underline py-2 {{ request()->is('/') ? 'text-yellow-400 font-bold' : '' }}">Home</a>
<a href="{{ url('/jobs') }}"
  class="text-white hover:underline py-2 {{ request()->is('jobs') ? 'text-yellow-400 font-bold' : '' }}">All
  Jobs</a>
<a href="{{ url('/jobs/saved') }}"
  class="text-white hover:underline py-2 {{ request()->is('jobs/saved') ? 'text-yellow-400 font-bold' : '' }}">Saved
  Jobs</a>
<a href="{{ url('/login') }}"
  class="text-white hover:underline py-2 {{ request()->is('login') ? 'text-yellow-400 font-bold' : '' }}">Login</a>
<a href="{{ url('/register') }}"
  class="text-white hover:underline py-2 {{ request()->is('register') ? 'text-yellow-400 font-bold' : '' }}">Register</a>
<a href="{{ url('/dashboard') }}"
  class="text-white hover:underline py-2 {{ request()->is('dashboard') ? 'text-yellow-400 font-bold' : '' }}">
  <i class="fa fa-gauge mr-1"></i> Dashboard
</a>
```

@class Directive

There is another way that we can do this using a `@class` directive. I am not going to keep this method because of the way we're going to do things in future videos with attributes and props, but I want to show you how to use it.

We can also use the `@class` directive like so:

```
<a href="{{ url('/') }}" @class([
  'text-white hover:underline py-2' ,
  'font-bold text-yellow-400'=>request()->is('/')
])>Home</a>

<a href="{{ url('/jobs') }}" @class([
```

```

        'text-white hover:underline py-2' ,
        'font-bold text-yellow-400'=> request()->is('jobs')
    ]>All Jobs</a>

    <a href="{{ url('/jobs/saved') }}" @class([
        'text-white hover:underline py-2' ,
        'font-bold text-yellow-400'=>request()->is('jobs/saved')
    ]>Saved Jobs</a>

    <a href="{{ url('/login') }}" @class([
        'text-white hover:underline py-2' ,
        'font-bold text-yellow-400'=> request()->is('login')
    ]>Login</a>

    <a href="{{ url('/register') }}" @class([
        'text-white hover:underline py-2' ,
        'font-bold text-yellow-400'=>request()->is('register')
    ]>Register</a>

    <a href="{{ url('/dashboard') }}" @class([
        'text-white hover:underline py-2' ,
        'font-bold text-yellow-400'=>request()->is('dashboard')
    ]>
        <i class="fa fa-gauge mr-1"></i> Dashboard
    </a>

```

So the `@class` directive takes in an array of classes. The first element is the default class. Then you can add additional classes with the key being the class and the value being the condition.

It is completely up to you on which method you want to use but for this project, we will be using the ternary operator.

8 - Component Attributes & Props

In this lesson, we are going to look at component attributes and props. We will create a `NavLink` component that accepts attributes and props. The difference between attributes and props is that attributes are for actual defined HTML attributes, while props are for custom attributes.

We have our navigation links styled and working as expected. However, we can make our navigation links more dynamic by creating a `NavLink` component that accepts attributes.

Run the following command to create a new component:

```
php artisan make:component NavLink
```

This creates a new component in the `View/Components` directory. Open the `nav-link.blade.php` file and replace the content with the following for now:

```
<a>
  {{ $slot }}
</a>
```

The `$slot` variable is a special variable that contains the content passed to the component. In this case, it will contain the text of the link.

So in the `header.blade.php` file, replace the home link with the following:

```
<x-nav-link href="{{ url('/') }}">Home</x-nav-link>
```

You should see the text "Home" displayed on the page. However, the link is not working yet. Let's update the `NavLink` component to include the `href` attribute.

Attributes

We can use the `$attributes` variable to pass all attributes to the component. This allows us to pass any attribute to the component without explicitly defining them.

Open the `nav-link.blade.php` file and update it as follows:

```
@php
echo $attributes;
@endphp

<a>
  {{ $slot }}
</a>
```

You should see the `href` attribute printed on the page and the class attribute.

Get rid of the `@php` block. I just wanted to show you how to use it.

Now simply add the `@attributes` variable to the `a` tag:

```
<a {{ $attributes }}>
  {{ $slot }}
</a>
```

This is different from `props`. We will look at props soon, but attributes are for actual defined HTML attributes, while props are for custom attributes. It should now work as expected.

Now one of my reasons for using a nav link component is so we don't have to pass so many attributes to the `a` tag. But now we are passing the same number of attributes to the `x-nav-link` component. We can move the class attribute to the `NavLink` component.

Open the `nav-link.blade.php` file and update it as follows:

```
<a {{$attributes}} class="text-white hover:underline py-2 {{ request()->is('/') ?  
'text-yellow-400 font-bold' : '' }}">  
    {{$slot}}  
</a>
```

However, now, it always checks for the homepage.

Props

We can pass custom attributes to the component using props. Let's create a prop called `active` that will determine if the link is active.

We need to define any custom props using the `@props` directive at the top of the component file. Open the `nav-link.blade.php` file and update it as follows:

```
@props(['active'])  
  
<a {{$attributes}} class="text-white hover:underline py-2 {{ request()->is('/') ?  
'text-yellow-400 font-bold' : '' }}">  
    {{$slot}}  
</a>
```

We are defining a prop called `active` that we can use in the component. By doing this, if we pass in the `active` prop, it will not be included in the `$attributes` variable.

Default Values

We can also give it a default value. Let's set the default value of `active` to `false`. Update the `@props` directive as follows:

```
@props(['active' => false])
```

Now, let's check for the `active` prop and add the `font-bold` and `text-yellow-400` classes if it is true. Update the `a` tag as follows:

```
<a {{$attributes}} class="text-white hover:underline py-2 {{ $active ? 'text-  
yellow-400 font-bold' : '' }}">  
    {{$slot}}  
</a>
```

Using Props

Now, in the `header.blade.php` file, we can pass the `active` prop to the `NavLink` component. Update the home link as follows:

```
<x-nav-link href="{{ url('/') }}" active="false">Home</x-nav-link>
```

Notice that the link is active now even though we passed `false`. This is because the `"false"` is being passed as a string, which will evaluate to `true`. To fix this, we need to pass a boolean value. Update the `active` prop as follows:

```
<x-nav-link href="{{ url('/') }}" :active="false">Home</x-nav-link>
```

We just added the `:` before the prop to pass it as a boolean. Now the link should not be active. If you change it to `true`, the link will be active. The `:` is not just for boolean values; it is for any dynamic value.

Now instead of passing true or false, pass the result of the `request()->is('/')` function. Update the `active` prop as follows:

```
<x-nav-link href="{{ url('/') }}" :active="request()->is('/')">Home</x-nav-link>
```

URL Prop

You could leave it like this, but I actually want to pass in the url as a prop because we may want to use the url in other ways in the future and this will make it easier. I also just think it looks cleaner.

Update the `@props` directive as follows:

```
@props(['url' => '/', 'active' => false])
```

Now edit the `a` tag to use the `url` prop:

```
<a href="{{ $url }}" class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold' : '' }}">
    {{ $slot }}
</a>
```

Now we can simply change all of our `<nav-link>` components to use the `url` prop. Update the links in the `header.blade.php` file as follows:

```
<x-nav-link url="/" :active="request()->is('/')">Home</x-nav-link>
<x-nav-link url="/jobs" :active="request()->is('jobs')">All Jobs</x-nav-link>
<x-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')">Saved Jobs</x-
```

```

nav-link>
<x-nav-link url="/login" :active="request()->is('login')">Login</x-nav-link>
<x-nav-link url="/register" :active="request()->is('register')">Register</x-nav-link>
<x-nav-link url="/profile" :active="request()->is('profile')"><i class="fa fa-user mr-1"></i> Profile
</x-nav-link>

```

Let's leave the create job link along because we are going to have a separate component for button links.

Icon Prop

Open the `views/layout.blade.php` file and let's cut the `<i class='fa fa-gauge'></i>`. Now paste it in the `views/components/nav-link.blade.php` file right before the `{{ $slot }}`:

```

<a
  href="{{ $url }}"
  class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold' : '' }}"
>
  <i class="fa fa-gauge mr-1"></i> {{ $slot }}
</a>

```

Let's add a new prop at the top of the `views/components/nav-link.blade.php` file:

```
@props(['url' => '/', 'active' => false, 'icon' => null])
```

We set the default value of the `icon` prop to `null`. Now replace the `<i class='fa fa-gauge'></i>` with the following code:

```

<a
  href="{{ $url }}"
  class="text-white hover:underline py-2 {{ $active ? 'text-yellow-400 font-bold' : '' }}"
>
  @if($icon)
    <i class="fa fa-{{ $icon }} mr-1"></i>
  @endif {{ $slot }}
</a>

```

Now you can pass in an icon prop to the `nav-link` component. Let's test it out.

Open the `views/header.blade.php` file and add the gauge icon to the dashboard link:

```
<x-nav-link url="/dashboard" :active="request()->is('dashboard')" icon="gauge"
  >Dashboard</x-nav-link
>
```

Now you should see the gauge icon next to the dashboard link.

9 - Button Link Component Challenge

In this challenge, I want you to create a new component called `ButtonLink` and use it for the `Create Job` link that is formatted as a button.

This component should get passed in the following props:

- `url` - URL the button link goes to
- `icon` - Optional icon
- `bgClass` - Class for the background. Default: `'bg-yellow-500'`
- `hoverClass` - Class for the hover color. Default: `'hover:bg-yellow-600'`
- `textClass` - Class for the text color. Default: `'text-black'`

When you embed the button link component, it should look like this:

```
<x-button-link url="/jobs/create" type="button" icon="edit">Create Job</x-button-link>
```

You should be able to pass in a class for the background, hover and text. If not, it uses the defaults that you describe in the `@props` directive.

► Click For Solution

Let's create a button component. Run the following command:

```
php artisan make:component ButtonLink
```

Let's add the props at the top of the `views/components/button-link.blade.php` file:

```
@props([
  'url' => '/',
  'icon' => null,
  'bgClass' => 'bg-yellow-500',
  'hoverClass' => 'hover:bg-yellow-600',
  'textClass' => 'text-black'
])
```

Just like the nav link, we have the url and icon. I also added the `bgClass`, `hoverClass`, and `textClass` props so we can change the background color, hover color, and text color of the button. Now add the following code to the `views/components/button-link.blade.php` file:

```
<a href="{{ $url }}"
    class="{{ $bgClass }} {{ $hoverClass }} {{ $textClass }} px-4 py-2 rounded
    hover:shadow-md transition duration-300">
    @if($icon)
    <i class="fa fa-{{ $icon }}" mr-1"></i>
    @endif
    {{ $slot }}
</a>
```

Now you can use the button link component to create a button link. Let's test it out.

In the `views/components/header.blade.php` file, replace the create job link with the following:

```
<x-button-link url="/jobs/create" icon="edit">Create Job</x-button-link>
```

That's it for our nav links. We will get to the mobile nav links in the next lesson.

10 - Mobile Menu Link

So we have the regular menu all set. Let's take care of the links in the mobile menu. These links classes differ a bit, so we can't just use the NavLink component as is. What I want to do is pass in a prop of `:mobile="true"` if it is a mobile link and it will then apply the correct classes.

Before we do anything though, we need to be able to see the mobile menu. We can not yet because we have not added the frontend JavaScript to toggle the mobile menu. We will do that in the next lesson. For now, let's remove the `hidden` class from the mobile menu in the header.

Remove the `hidden` and `md:hidden` class from the following code in the `header.blade.php` file:

```
<nav
  div
  id="mobile-menu"
  class="hidden md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
></nav>
```

Open the `components/nav-link.blade.php` file and add an extra prop for "mobile":

```
@props(['url' => '/', 'active' => false, 'icon' => null, 'mobile' => false])
```

Now add an if directive and add the mobile link classes:

```
@if($mobile)
<a href="{{ $url }}" class="block px-4 py-2 hover:bg-blue-700 {{$active ? 'text-yellow-500 font-bold' : '' }}">
    @if($icon)
    <i class="fa fa-{{$icon}}" mr-1"></i>
    @endif
    {{$slot}}
</a>
@else
<a href="{{ $url }}" class="text-white hover:underline py-2 {{$active ? 'text-yellow-500 font-bold' : '' }}">
    @if($icon)
    <i class="fa fa-{{$icon}}" mr-1"></i>
    @endif
    {{$slot}}
</a>
@endif
```

Now replace the mobile menu links in the header with the following:

```
<x-nav-link url="/jobs" :active="request()->is('jobs')" :mobile="true">All
Jobs</x-nav-link>
<x-nav-link url="/jobs/saved" :active="request()->is('jobs/saved')"
:mobile="true">Saved Jobs</x-nav-link>
<x-nav-link url="/login" :active="request()->is('login')" :mobile="true">Login</x-
nav-link>
<x-nav-link url="/register" :active="request()->is('register')"
:mobile="true">Register</x-nav-link>
<x-nav-link url="/dashboard" :active="request()->is('dashboard')"
:mobile="true">Dashbaord</x-nav-link>
```

Mobile Button Link

The only difference between the mobile button link and the ButtonLink component is that the mobile version has a class of `block`. So let's add a prop of `block` in the `components/button-link.blade.php` file:

```
@props([
'url' => '/',
'icon' => null,
'bgClass' => 'bg-yellow-500',
'hoverClass' => 'hover:bg-yellow-600',
'textClass' => 'text-black',
'block' => false
])
```

Now add a ternary checking for that prop and if it is true, show a class of `block`:

```
<a href="{{ $url }}"
  class="{{ $bgClass }} {{ $hoverClass }} {{ $textClass }} px-4 py-2 rounded
  hover:shadow-md transition duration-300 {{ $block ? 'block' : '' }}">
  @if($icon)
    <i class="fa fa-{{ $icon }}"></i>
  @endif
  {{ $slot }}
</a>
```

Now the mobile menu should look the same. It is just using components for the links and button.

Now add the `hidden` and `md:hidden` class back to the mobile menu in the `header.blade.php` file:

```
<nav
  id="mobile-menu"
  class="hidden md:hidden bg-blue-900 text-white mt-5 pb-4 space-y-2"
></nav>
```

In the next lesson, we will add the JavaScript to toggle the mobile menu as well as the image and css assets. We will also create the hero component.

11 - Mobile Menu Toggle

This is going to be a very quick and simple lesson. We want to make the toggle work. Now ultimately, when we need to add some interactive JavaScript, I would suggest using Alpine.js, which is a lightweight JavaScript framework that provides reactivity and declarative rendering in your HTML markup. So you don't even have to write any JavaScript for things like toggling content and modals. We'll be using this later, but for now we'll just have a few lines of vanilla JavaScript.

Create a file at `/public/js/script.js` and add the following JavaScript:

```
document.querySelector('#hamburger').addEventListener('click', function () {
  const menu = document.getElementById('mobile-menu');
  menu.classList.toggle('hidden');
});
```

This is for toggling the mobile menu.

asset Helper Function

We can include these files in the layout with the `asset` helper function. This function generates a URL for an asset file.

Open the `/resources/views/components/layout.blade.php` file and add the following line to the bottom of the body section:

```
<script src="{{ asset('js/script.js') }}"></script>
```

Your toggle button for the mobile menu should work now.

12 - Hero Component

Now let's work on the hero component. This will have a background image and a search form. The search form will be put into it's own component much later, so we won't worry about that just yet. I just want to get it displayed on the homepage.

Run the following command to generate a new component called `Hero`.

```
php artisan make:component Hero
```

Open the newly created `/resources/views/components/hero.blade.php` file and replace the content with the following:

```
<section
  class="hero relative bg-cover bg-center bg-no-repeat h-72 flex items-center"
>
  <div class="overlay"></div>
  <div class="container mx-auto text-center z-10">
    <h2 class="text-5xl text-white font-bold mb-8">Find Your Dream Job</h2>
    <form class="block mx-5 md:mx-auto md:space-x-2">
      <input
        type="text"
        name="keywords"
        placeholder="Keywords"
        class="w-full md:w-72 px-4 py-3 focus:outline-none"
      />
      <input
        type="text"
        name="location"
        placeholder="Location"
        class="w-full md:w-72 px-4 py-3 focus:outline-none"
      />
      <button
        class="w-full md:w-auto bg-blue-700 hover:bg-blue-600 text-white px-4 py-3
focus:outline-none"
      >
        <i class="fa fa-search mr-1"></i> Search
      </button>
    </form>
```

```
</div>
</section>
```

Like I said, the form will be its own component. We will handle that later.

Background Image

To show the background image, we need some custom CSS and we need to bring the image into the folder structure. Copy the file `_theme_files/images/hero.jpg` to `/public/images/hero.jpg`.

Create a file at `/public/css/style.css` and add the following CSS:

```
.overlay {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.8); /* Adjust opacity as needed */
  z-index: 1;
}

.hero {
  background-image: url('../images/hero.jpg');
}
```

Open the `/resources/views/components/layout.blade.php` file and add the following line to the head section:

```
<link rel="stylesheet" href="{{ asset('css/style.css') }}" />
```

Title Prop

Let's make the H2 text a prop that can be passed in. Add the following to the top of the file:

```
@props(['title' => 'Find Your Dream Job'])
```

Then replace the H2 text with the following:

```
<h2 class="text-4xl text-white font-bold mb-4">{{ $title }}</h2>
```

Limiting the Hero To The Homepage

We have an issue here. I only want the hero component on the homepage but remember all pages are wrapped in a `<main class="container mx-auto p-4 mt-4">` tag. This means that any component in the homepage will be restricted to the container. The hero needs to go all the way across the page. The solution is to put this component in the layout but limit it to the homepage by using the `request` helper function.

Open the `/resources/views/components/layout.blade.php` file and add the following code above the opening `main` tag:

```
@if(request()->is('/'))
<x-hero />
@endif
```

This will only show the hero component on the homepage.

13 - Top & Bottom Banner Components

We have a couple other components to create. We have the top banner on the homepage which goes right under the hero section. We also have the bottom banner, which will go on the homepage as well as some other pages.

Let's start with the top banner. Run the following command to create a new component:

```
php artisan make:component TopBanner
```

Let's add the following code to the `/resources/views/components/top-banner.blade.php` file:

```
<section class="bg-blue-900 text-white py-6 text-center">
  <div class="container mx-auto">
    <h2 class="text-3xl font-semibold">Unlock Your Career Potential</h2>
    <p class="text-lg mt-2">Discover the perfect job opportunity for you.</p>
  </div>
</section>
```

Now, add it to the `/resources/views/components/layout.blade.php` file and limit to the homepage just like the hero component:

```
@if(request()->is('/'))
  <x-hero />
  <x-top-banner />
@endif
```

Let's make the a couple props for the heading and subheading. Update the `TopBanner` component like so:

```
@props(['heading' => 'Unlock Your Career Potential', 'subheading' => 'Discover the
perfect job opportunity for you'])

<section class="bg-blue-900 text-white py-6 text-center">
  <div class="container mx-auto">
    <h2 class="text-3xl font-semibold">
      {{ $heading }}
    </h2>
    <p class="text-lg mt-2">
      {{ $subheading }}
    </p>
  </div>
</section>
```

Now if you wanted to, you can pass in the heading and subheading.

Bottom Banner Component

Now let's create the bottom banner component. Run the following command to create a new component:

```
php artisan make:component BottomBanner
```

Add the following code to the `/resources/views/components/bottom-banner.blade.php` file:

```
<section class="container mx-auto my-6">
  <div
    class="bg-blue-800 text-white rounded p-4 flex items-center justify-between"
  >
    <div>
      <h2 class="text-xl font-semibold">Looking to hire?</h2>
      <p class="text-gray-200 text-lg mt-2">
        Post your job listing now and find the perfect candidate.
      </p>
    </div>
    <a
      href="create-job.html"
      class="bg-yellow-500 hover:bg-yellow-600 text-black px-4 py-2 rounded
        hover:shadow-md transition duration-300"
    >
      <i class="fa fa-edit"></i> Create Job
    </a>
    </div>
</section>
```

We can also add a heading and subheading component here. Another thing that we can do, since it uses the same button as the header is we can use the `ButtonLink` component that we created earlier. Update the `BottomBanner` component like so:

```
@props(['heading' => 'Looking to hire?', 'subheading' => 'Post your job listing
now and find the perfect
candidate'])

<section class="container mx-auto my-6">
  <div class="bg-blue-800 text-white rounded p-4 flex items-center justify-
between">
    <div>
      <h2 class="text-xl font-semibold">{{$heading}}</h2>
      <p class="text-gray-200 text-lg mt-2">
        {{$subheading}}
      </p>
    </div>
    <x-button-link url="/jobs/create" type="button" icon="edit">Create Job</x-
button-link>
  </div>
</section>
```

Since the bottom banner does not need to span the whole page width, we can put it right in the home view. Update the `resources/views/pages/home.blade.php` file like so:

```
<x-layout>
  <x-bottom-banner />
</x-layout>
```

Now it should show on the homepage. We will add it to the other pages later.